

Bike Rental APP_Development

ZIPPI Rider Mobile App - Flutter + Laravel Backend

Build Plan, Architecture, Integrations & Delivery Schedule

Prepared for: ZIPPI

Platform: Android-first (iOS later) | Stack: Flutter (Dio, Riverpod) + Laravel 13 API

Integrations: Edumarc (OTP) - Quickkyc (Aadhaar/DL KYC) - Razorpay (payments)

Date: 24 June 2026 | Status: Backend wiring complete; app build next

1. Executive Summary

This document is the build plan for the ZIPPI bike-rental rider mobile app. The backend already exists: a Laravel 13 REST API at /api/rental/v1 with 42 rider endpoints, Sanctum token authentication, a full booking state machine, wallet, Razorpay payments, KYC scaffolding and notifications. The remaining work is (a) wiring two integrations you already own - Edumarc for OTP and Quickkyc for KYC - and (b) building the Flutter app against the existing API.

Key decisions

- **Scope:** Build the Flutter rider app AND wire the backend integrations.
- **KYC:** Auto Aadhaar + Driving Licence + selfie via Quickkyc; instant approval on pass, fallback to manual admin review on failure.
- **Platform:** Android-first; develop in VS Code / Cursor (Android Studio only for SDK + emulator).
- **Design:** The Claude Design sketch is the full-app design; the UI follows it.
- **Out of scope:** RC and Banking verification are vehicle/owner concerns - kept out of rider onboarding (reserved for a future hub/owner flow).

Outcome

A working Android app where a rider signs up with an Edumarc OTP, completes automated Aadhaar+DL+selfie KYC via Quickkyc, browses bikes, books, pays, and rides - all talking to the existing Laravel backend.

2. What Already Exists (Backend)

The repository ZIPPI-RENTAL-MVP is a production-structured Laravel 13 monolith. The rider app does not need new backend features for the core loop - only the two provider swaps below.

Capability	Status
Rider REST API (/api/rental/v1) - 42 endpoints	Built & tested
Sanctum bearer-token auth (mobile + OTP)	Built
Booking state machine (quote->hold->pay->active->return)	Built
Razorpay payments + idempotency	Built
Wallet, refunds, notifications	Built
OTP delivery	Was MSG91 -> now Edumarc-ready
KYC verification	Was manual stub -> now Quickkyc-ready
Admin dashboard + Hub Operations module	Built

Auth model

Riders authenticate with mobile + OTP and receive a Laravel Sanctum bearer token. Send 'Authorization: Bearer <token>' on every protected request. Money is always integer paise - divide by 100 only at render time.

3. Backend Wiring - COMPLETED

Both integrations are implemented behind the existing adapter pattern, are selectable by config, default to safe values (no behavior change until env vars are set), and are covered by tests. The full backend suite passes: 124 tests.

3A. Edumarc OTP (swap from MSG91)

- New **EdumarcProvider** implements the **OtpProvider** contract (modelled on **Msg91Provider**).
- **OTP provider is config-selectable: `OTP_PROVIDER = log | edumarc | msg91` (default log).**
- **Config + .env keys added: `EDUMARC_API_KEY`, `EDUMARC_SENDER_ID`, `EDUMARC_TEMPLATE_ID`, `EDUMARC_ENDPOINT`, `EDUMARC_MESSAGE_TEMPLATE`.**

To go live: set `OTP_PROVIDER=edumarc` and the `EDUMARC_*` values in the production .env; confirm Edumarc's exact request payload against your account and adjust if field names differ.

3B. Quickkyc automated KYC

- New **QuickkycProvider** implements the **KycProvider** contract; selectable via **`KYC_PROVIDER = auto | quickkyc`** (default auto = manual review, unchanged).
- **Contract extended so the raw document number + DOB reach the provider for number-based verification (only the masked number is persisted).**
- **KycService now honours the provider result: on submit, verified required docs -> instant auto-approve; a failed doc -> auto-reject; otherwise -> manual review.**

Important caveat (open item)

Quickkyc's Driving-Licence check is a synchronous number+DOB lookup, so it auto-verifies now. Aadhaar is OTP-based (the holder receives an OTP) and cannot complete in a single call, so Aadhaar/selfie return 'pending' and route to review until the Aadhaar-OTP sub-flow is wired with Quickkyc's exact contract. Endpoint and response-field mapping are config-driven (`QUICKKYC_*`), so matching them needs no code change.

4. Where to Build & First Steps

Where: VS Code / Cursor (not Android Studio for daily work)

- Install Android Studio once - only to get the Android SDK, platform-tools and an emulator (AVD). You will rarely open it afterwards.
- Code day-to-day in VS Code or Cursor with the Flutter extension (bundles Dart).
- Create the Flutter app as a separate project from the Laravel backend (e.g. a sibling folder or a mobile/ subfolder) so deploys stay independent.

The very first step: get Flutter talking to the API

- Install toolchain: brew install --cask flutter + Android Studio, then run 'flutter doctor' until Android shows OK.
- Create the app: flutter create --org in.zippi rider_app; open in Cursor; 'flutter run' on the emulator (blank app launches).
- Run the backend locally: php artisan serve (serves on http://127.0.0.1:8000).
- Link the app: the Android emulator maps the host to 10.0.2.2, so set API_BASE_URL = http://10.0.2.2:8000/api/rental/v1.
- Prove the link: call GET catalog/categories (public, no auth) and render the JSON. Real data on screen = the app is officially linked to the backend.
- Physical phone testing: use your Mac's LAN IP and run php artisan serve --host=0.0.0.0.

5. App Foundations (build once)

Everything depends on these. ETA ~3.25-4.25 dev-days.

- **Scaffold:** Flavors dev/staging/prod, each with its own API_BASE_URL.
- **API client:** Dio with base URL, auth header, Idempotency-Key on POST bookings & POST payments/verify, and an error mapper (401->login, 403-KYC->KYC, 422->fields, 5xx->retry).
- **Session:** flutter_secure_storage for the Sanctum token; auth-state stream the router listens to.
- **Money utils:** API returns integer paise - formatPaise(12500) -> Rs.125.00; never do float math on money.
- **Navigation:** Named routes, auth guard, KYC-gate flag, bottom-nav shell.
- **State:** Riverpod (or Bloc); model the booking flow as an explicit state machine mirroring the backend BookingStatus.

6. Screens & Sections (mapped to the design)

Build order follows the critical path A -> B -> C -> D -> E. Each section uses the same shape: Purpose -> UI -> logic -> API -> done-when. Visuals come from the Claude Design.

Section	Scope	ETA (days)
B. Auth & onboarding	Splash, phone entry, OTP (Edumarc), signup -> KYC	3.25-3.75
C. KYC (Quickkyc)	Aadhaar# + DL#+DOB entry, selfie capture, instant verify, status banner	2.5-3
D. Browse & discovery	Dashboard, catalog, available-bikes (date filter), hub map + helmet add-on	4.5-5.5
E. Booking & payment	Quote -> hold (3-min) -> Razorpay -> verify -> details; unlock/return/cancel	5.5-6.5
F. Post-ride & ambient	History, wallet, notifications (FCM), support, profile/settings	4.25-4.75
G. Cross-cutting	Deposit display, speed-limit terms, geofence (optional v2)	3-4

Notes vs. your integrations

- Section B OTP screen reflects Edumarc delivery (4-6 digit boxes, resend timer using the backend's 30s cooldown / 5-attempt rules).
- Section C changes from 'upload images' to 'enter Aadhaar/DL number + DOB + take selfie -> instant result' because of automated Quickkyc.
- Section E is the highest-risk section (money, holds, idempotency) - budget extra QA.
- MVP de-scope: drop geofence (G3) to v2 and keep Support as a simple link - saves ~4-5 dev-days and pulls the core loop forward.

7. Delivery Schedule & ETA

Track	ETA (dev-days)
Backend wiring (Edumarc + Quickkyc) - DONE	2.5-3.5
App foundations	3.25-4.25
Screens B-G	23-27.5
QA / device matrix (+25%)	~7-8
Design polish pass (+15%)	~4-4.75
Play Store submission & release	2-3
GRAND TOTAL (1 engineer)	~42-51 dev-days

Calendar: ~8-10 weeks for one engineer; ~5-6 weeks with two (one on A/B/C, one on D/E after foundations land). Backend wiring can run in parallel on day 1 (already complete).

Sprint sequence (2-week sprints)

- S1 - Foundations + Auth working end-to-end (incl. Edumarc).
- S2 - KYC (Quickkyc) + browse/discovery.
- S3 - Core booking loop (quote -> hold -> pay -> details).
- S4 - Wallet, history, notifications, cross-cutting.
- S5 - Hardening, QA, polish, store submission.

8. Starter Prompts (for Cursor / Claude)

Prompt 0 - project setup (after flutter create)

"This is a Flutter app for a bike-rental rider client that talks to a Laravel REST API at /api/rental/v1 (Sanctum bearer-token auth, all money in integer paise). Set up: flavor configs for dev/staging/prod with an API_BASE_URL each (dev = http://10.0.2.2:8000/api/rental/v1); a Dio HTTP client with an auth-header interceptor, an Idempotency-Key interceptor for POST /bookings and POST /payments/verify, and an error mapper (401->login, 403-KYC->KYC, 422->field errors, 5xx->retry-with-backoff); flutter_secure_storage for the token; Riverpod for state; a money util formatPaise. Generate the folder structure and these foundation files only."

Prompt 1 - a screen (repeat per screen, paste the design)

"Build the <ScreenName> screen to match this design [attach the Claude Design screenshot]. It calls <METHOD path> from our API. Purpose: <...>. On success <...>, on error map per our interceptor. Use our existing Dio client and Riverpod providers. Money fields are paise - render with formatPaise. Add the route to the navigation shell."

Prompt 2 - backend Edumarc adapter

"Create app/Integrations/Otp/EdumarcProvider.php implementing the OtpProvider contract, modelled on Msg91Provider, that sends OTP SMS via Edumarc using EDUMARC_API_KEY, EDUMARC_SENDER_ID, EDUMARC_TEMPLATE_ID. Bind it for staging/prod, keep LogOtpProvider for local. Add a feature test." (DONE)

Prompt 3 - backend Quickkyc adapter

"Create app/Integrations/Kyc/QuickkycProvider.php implementing KycProvider, calling Quickkyc's Aadhaar and Driving-Licence verification (number-based). Return auto_result = verified/failed/pending. Update KycService to honour auto_result. Add env keys QUICKKYC_* and feature tests." (DONE)

9. Verification (end-to-end)

- Backend suite: php artisan test - currently 124 passing (incl. new Edumarc & Quickkyc tests).
- OTP path: on staging request an OTP -> a real Edumarc SMS arrives -> verify-otp returns a token.
- KYC path: valid DL -> auto-approved; invalid -> manual review; POST bookings blocked (403) until approved.
- App link smoke test: fresh emulator calls GET catalog/categories and renders data.
- Core loop: sign up -> KYC -> browse -> quote -> hold -> Razorpay test mode -> verify -> confirmed -> unlock -> return -> deposit refund in wallet. Re-send POST bookings with the same Idempotency-Key -> no duplicate booking.
- Release check: flutter build apk --flavor prod succeeds; install on a physical device pointing at staging.

Open items to confirm

- Edumarc API endpoint + DLT template id (for live OTP).
- Quickkyc Aadhaar flow type (OTP-based vs offline XML) and exact request/response fields.
- Razorpay keys (test + live) and the checkout SDK key for the app.
- FCM project for push notifications.